

LearningSteps: Origins

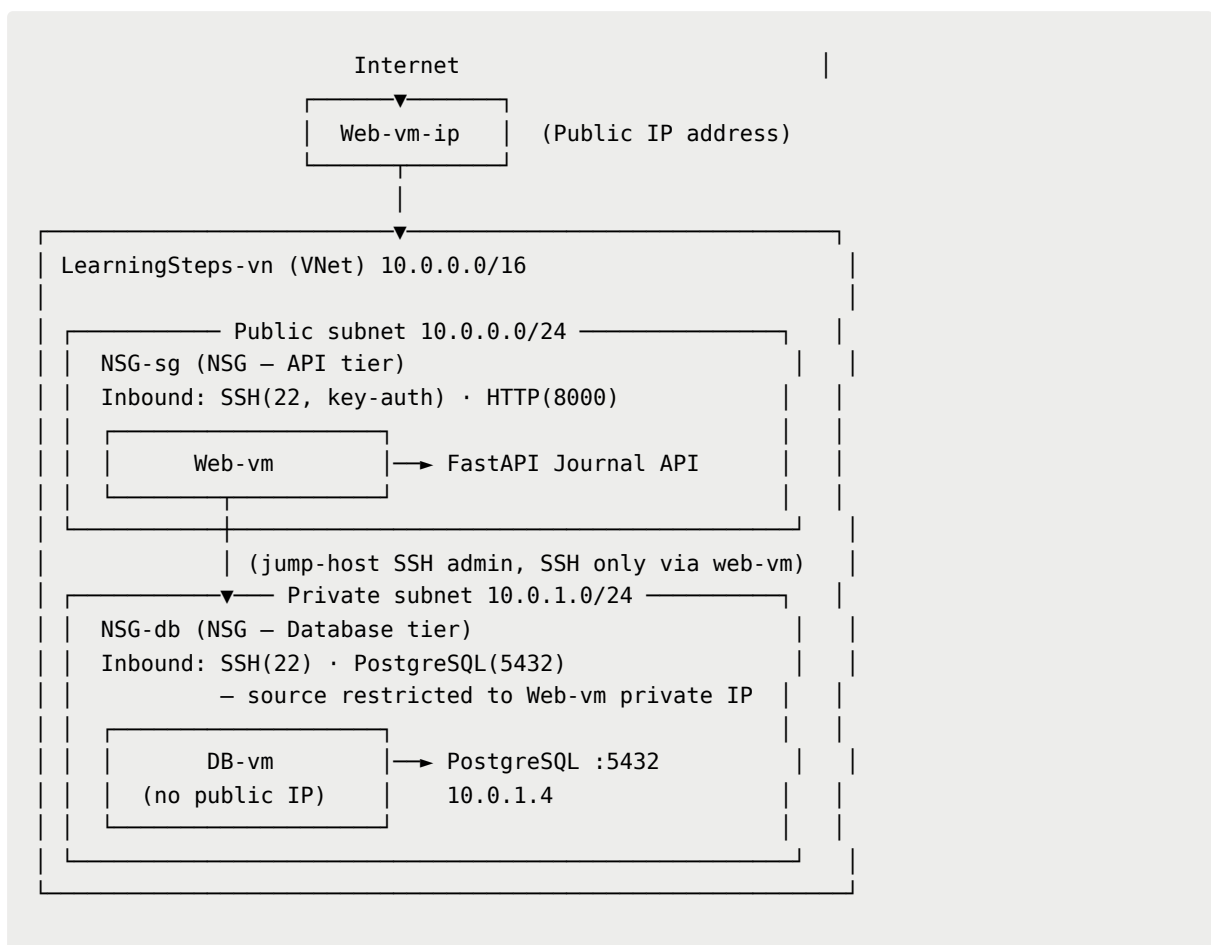
A 2-Tier Cloud Architecture on Azure

Training Program by: Cybersteps

Cohort: 12/25

Name: E-Itohan

Date: 04.07.2026



Architecture schema

1. Objective

Deploy the LearningSteps journal API (Python FastAPI + PostgreSQL) on Azure using a **two-tier architecture**, with the database isolated from the public internet and reachable only from the application tier within the virtual network.

2. Architecture Overview

The deployment follows a classic **public/private subnet segmentation** model on Azure:

☐	Name ↑	IPv4	IPv6	Available IPs	Delegated to	Security group
☐	Public	10.0.0.0/24	-	250	-	NSG-sg
☐	Private	10.0.1.0/24	-	250	-	db-sc

Subnets

- **Virtual Network (VNet):** 10.0.0.0/16 , containing two subnets:
 - **Public subnet** (10.0.0.0/24) — hosts the web/API VM with a public IP, exposed to inbound traffic on the required ports only.

Inbound Security Rules						
100	AllowSSH	22	Any	69.16.145.172	Any	✓ Allow
110	AllowHTTP	8000	Any	Any	Any	✓ Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	✓ Allow
65001	AllowAzureLoadBalan...	Any	Any	AzureLoadBalancer	Any	✓ Allow
65500	DenyAllInBound	Any	Any	Any	Any	✗ Deny

Public security rules

- **Private subnet** (10.0.1.0/24) — hosts the PostgreSQL VM. **No public IP assigned.** The database is reachable exclusively from inside the VNet.

100	AllowConnectionPostg...	5432	Any	10.0.0.0/24	Any	✓ Allow
110	AllowSSH-DB	22	Any	10.0.0.0/24	Any	✓ Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	✓ Allow
65001	AllowAzureLoadBalan...	Any	Any	AzureLoadBalancer	Any	✓ Allow
65500	DenyAllInBound	Any	Any	Any	Any	✗ Deny

Private security rules

- **Jump host pattern:** Since the DB VM has no public exposure, it is administered by first SSH-ing into the web VM, then hopping to the DB via

its private address. Access is effectively chained and controlled at each layer.

Search subnets

☐ Name ↑

☐ Public

☒ Private

IPv6

Include an IPv6 address space ☐ This virtual network has IPv6 enabled ☐

Private subnet

Private subnets enhance security by not providing default outbound access to the internet, it is necessary to explicitly grant outbound access. A NAT gateway is required for virtual machines in the subnet. [Learn more](#)

Enable private subnet (no default outbound access) ☒

i After March 31, 2026, private subnet will be the default selection for new virtual networks.

Security

Simplify internet access for virtual machines by using a network address group. [Learn more](#)

Showing 2 subnets

Save **Cancel**

Private subnet isolation on the portal

```
PS C:\Users\Pupila> ssh -i Documents\az_keys\Webvm_key833.pem azureuser@4.182.26.211
The authenticity of host '4.182.26.211 (4.182.26.211)' can't be established.
ED25519 key fingerprint is SHA256:nIhECgNkutKzZ79SQCbzKkVVopB7hzAdiBARzfRWKKU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '4.182.26.211' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1018-azure x86_64)
```

Web VM connection from host

```

azureuser@Web-vm:~$ ssh azureuser@10.0.1.4
The authenticity of host '10.0.1.4 (10.0.1.4)' can't be established.
ED25519 key fingerprint is SHA256:ySMagN7hcLtX0ALdZaCmRlPhABYgeSqVvlxGDStpNQE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.1.4' (ED25519) to the list of known hosts.
azureuser@10.0.1.4's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1018-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Jul  3 13:21:04 UTC 2026

System load:  1.19           Processes:            144
Usage of /:   6.1% of 28.02GB Users logged in:          0
Memory usage: 4%            IPv4 address for eth0: 10.0.1.4
Swap usage:   0%

Expanded Security Maintenance for Applications is enabled.

0 updates can be applied immediately.

Currently attempting to automatically attach this machine to an Ubuntu Pro subscription

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@DB-vm:~$ |

```

DB VM connection from Web VM

3. Implementation Steps

3.1 Database Server (Private Tier)

After connecting to the DB VM through the web jump host (`ssh azureuser@10.0.1.4`), PostgreSQL 16 was installed and configured:

```

CREATE DATABASE learning_journal;
CREATE USER <app_user> WITH ENCRYPTED PASSWORD '<secret>';
GRANT ALL PRIVILEGES ON DATABASE learning_journal TO <app_user>;

CREATE TABLE IF NOT EXISTS entries (
    id VARCHAR PRIMARY KEY,
    data JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_entries_created_at ON entries(created_at);
CREATE INDEX IF NOT EXISTS idx_entries_data_gin ON entries USING GIN (data);

```

```
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO <app_user>;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO <app_user>;
```

Notes:

- The JSONB column with a **GIN index** allows efficient querying inside the `data` document matching the API's flexible schema.
- Privileges were granted on the schema objects (tables/sequences) in addition to the database, because `GRANT ... ON DATABASE` alone does not confer table access in PostgreSQL.
- PostgreSQL was enabled for auto-start on reboot: `sudo systemctl enable postgresql.service`.

```
azureuser@DB-vm:~$ sudo systemctl enable postgresql.service
Synchronizing state of postgresql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable postgresql
```

Remote access configuration: two files were modified so the DB accepts connections from the app server only:

- `pg_hba.conf`: added an entry authorizing the **web server's private IP** for the app user and database.

```
# Database administrative login by Unix domain socket
local all postgres peer

# TYPE DATABASE USER ADDRESS METHOD
# "local" is for Unix domain socket connections only
local all all peer
# IPv4 local connections:
host all all 127.0.0.1/32 scram-sha-256
host all all 10.0.0.0/24 md5
# IPv6 local connections:
host all all ::1/128 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all peer
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
```

pg_hba.conf

- `postgresql.conf`: set `listen_addresses = '*'` so PostgreSQL listens on all interfaces — safe *only* because the NSG restricts who can reach port 5432.

```

#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
port = 5432                     # (change requires restart)
max_connections = 100           # (change requires restart)
#reserved_connections = 0       # (change requires restart)
#superuser_reserved_connections = 3 # (change requires restart)
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories

```

postgresql.conf

Connectivity was verified from the web VM with `nc -v <db-private-ip> 5432`.

```

azureuser@Web-vm:~$ nc -v 10.0.1.4 5432
Connection to 10.0.1.4 5432 port [tcp/postgresql] succeeded!

```

Connection verification

3.2 Application Server (Public Tier)

The reference implementation was fetched from the [upstream repository](#), the environment configured via `.env` (with cloud-appropriate values pointing to the DB's **private** IP), dependencies installed, and the app started with `./start.sh`.

```

# Copy this file to .env (which is git-ignored) and adjust as needed.
# This will ensure that you do not commit sensitive information to version control.
# This URL points to the local PostgreSQL container started via docker-compose.

POSTGRES_USER=user123
POSTGRES_PASSWORD=password123
POSTGRES_DB=learning_journal

DATABASE_URL=postgresql://postgres:postgres@postgres:5432/learning_journal

```

Environments configuration

LearningSteps API 0.1.0 OAS 3.1

/openapi.json

A simple learning journal API for tracking daily work, struggles, and intentions

default

GET	/entries	Get All Entries	⌵
POST	/entries	Create Entry	⌵
DELETE	/entries	Delete All Entries	⌵
GET	/entries/{entry_id}	Get Entry	⌵
PATCH	/entries/{entry_id}	Update Entry	⌵
DELETE	/entries/{entry_id}	Delete Entry	⌵

Web app

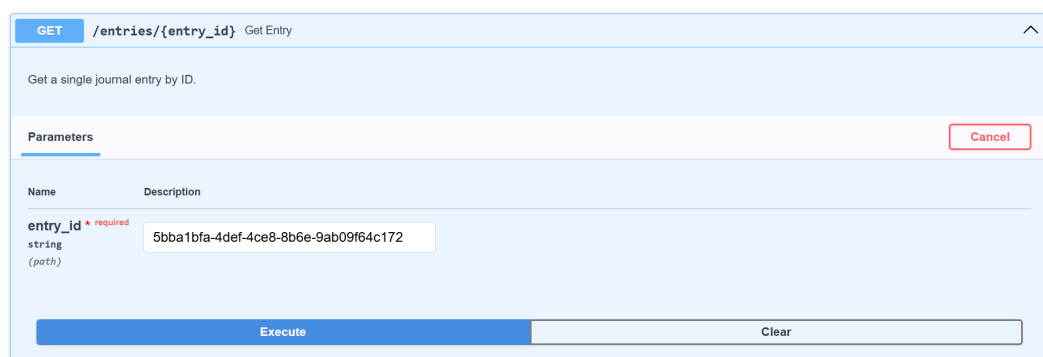
3.3 Deployment & Verification

- **POST** requests created journal entries (sample entry submitted via the API).

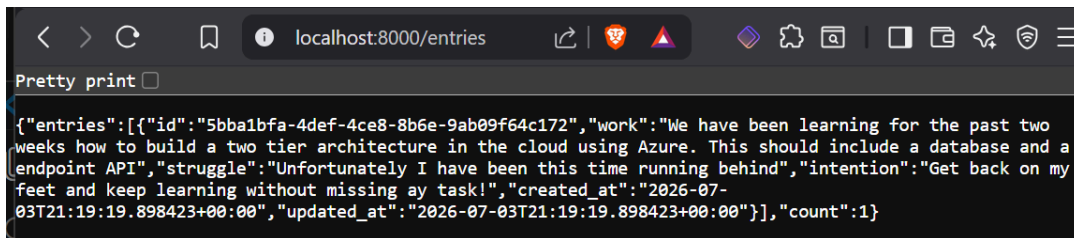


POST entry value

- **GET** requests retrieved stored entries, confirming the end-to-end path: **API** → **VNet** → **PostgreSQL**.



GET execution



```
localhost:8000/entries
Pretty print
{"entries":[{"id":"5bba1bfa-4def-4ce8-8b6e-9ab09f64c172","work":"We have been learning for the past two weeks how to build a two tier architecture in the cloud using Azure. This should include a database and a endpoint API","struggle":"Unfortunately I have been this time running behind","intention":"Get back on my feet and keep learning without missing ay task!","created_at":"2026-07-03T21:19:19.898423+00:00","updated_at":"2026-07-03T21:19:19.898423+00:00"}],"count":1}
```

GET verification

- Database unreachability from the public internet was validated by the absence of any public IP/endpoint on the DB VM, plus NSG enforcement.

4. Security Decisions

Decision	Rationale
Two subnets (public/private)	Segmentation. Compromising the web tier does not grant direct internet-facing access to the database; an attacker would need to pivot through the VNet.
DB VM has no public IP	The database is unreachable from the internet by construction , no firewall misconfiguration can accidentally expose it.
Private-subnet NSG: inbound 5432 allowed only from the web VM's private IP ; SSH only from the web tier	Least privilege. Only the application needs database access, and only admins (via jump host) need shell access.
Public-subnet NSG: inbound restricted to SSH (key-based) and the API port	Minimizes attack surface. Key-based SSH removes password brute-force risk.
Dedicated DB user with limited privileges	The app doesn't run as the <code>postgres</code> superuser. This limits damage if the app is compromised (SQL injection → contained blast radius).
Secrets in <code>.env</code> (git-ignored) only	Prevents credentials from entering version control history.

Trade-off acknowledged: enabling `listen_addresses = '*'` is broader than strictly needed, but acceptable here because network-layer controls (NSG) provide the actual boundary. A stricter alternative would be pinning it to the DB VM's private IP.

5. Success Criteria

Criterion	Status
API reachable via public IP	✓ POST/GET tested successfully
DB unreachable from public internet	✓ No public IP, NSG-scoped
CRUD via API endpoints	✓ Entries created and retrieved
Data persistence	✓ PostgreSQL on dedicated VM with enabled service; survives API reboot
Least-privilege NSGs	✓ Scoped to specific IPs/ports

6. Challenges

1. **Git workflow with upstream branches** → fetching the `reference` branch from upstream and tracking it correctly.

7. Key Learnings

- A **resource group** in Azure is the organizational boundary. Every VNet, NIC, IP, and VM was created inside one, simplifying lifecycle management (delete the RG = delete the whole stack).
- Azure security enforcement happens at **NSG** level (subnet and/or NIC), complementing OS-level controls like `pg_hba.conf`. Defense in depth means both must agree.
- **Public vs. private IP placement determines reachability** before any firewall rule is written.
- The **jump host pattern** trades convenience for security. This is acceptable for administration, not for production apps.